

UNIVERSITY OF LONDON INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects



CM3070 PROJECT FINAL PROJECT REPORT

Neural Style Transfer: A Comparative Study of Backbone, Layer Selection and Loss Weighting

Github:

<https://github.com/ererrorer/Neural-Style-Transfer-A-Comparative-Study-of-Backbone-Layer-Selection-and-Loss-Weighting>

Author: Er Bing Yang, Melvin

Student Number :190429524

Date of Submission: 28th September 2026

Supervisor :Dr Andrew Yoong

1 Introduction (876 words)

1.1 Project Concept

This project builds and evaluates a Neural Style Transfer (NST) system. NST is a deep learning method that generates an image combining the content of one photograph with the visual style of another, for example rendering a photograph of a city street in the brushwork of Van Gogh’s *Starry Night*. The core of the project is an end-to-end style transfer pipeline built around pre-trained convolutional networks. Around this core, the project adds a comparison study examining how the choice of network, its internal settings, and its optimisation method shape the output produced.

The project follows Project Idea 3.1, Neural Style Transfer, from the CM3015 Machine Learning and Neural Networks template, and uses Chollet’s *Deep Learning with Python* [1] as a baseline for code reference and background reading. The template directs the work toward a comparison across network architectures and settings, supported by both qualitative and quantitative evaluation, which shapes the structure of the investigation that follows.

1.2 Motivation and Existing Neural Style Transfer Applications

Neural Style Transfer is one of the few deep learning techniques to reach mass consumer use. Jing et al. (2019), in their review of the field, point to the commercial success of mobile applications such as Prisma, which applied the method introduced by Gatys, Ecker, and Bethge (2016) to convert everyday photographs into stylised artwork. The application reached the top of app store rankings in several countries shortly after release, an outcome the review cites as evidence of public demand for accessible style transfer tools.

Applications such as Prisma are deliberately closed, as illustrated in Figure 1.

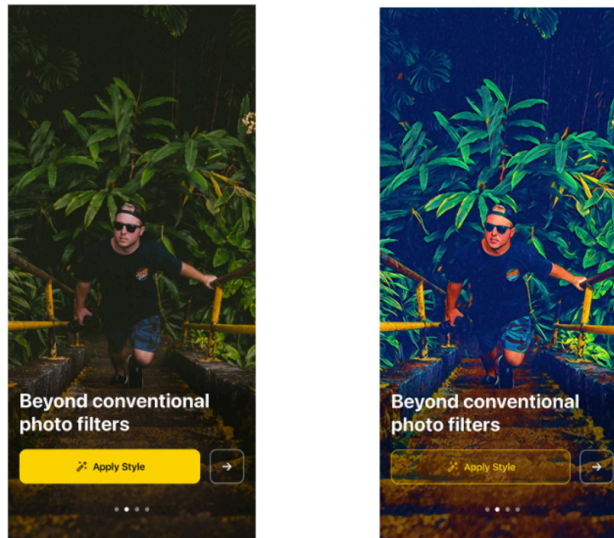


Figure 1: The Prisma mobile application, showing style transfer applied to a photograph with no user-facing controls over the underlying network or weighting parameters. Source: Prisma App.

A user selects a style, taps once, and receives a finished image with no view of how it was produced and no way to influence the process. The underlying network, the layers used to separate content from style, and the balance struck between the two are all fixed choices made in advance, hidden from the user rather than offered as adjustable settings. For a casual user this is generally acceptable, since the goal is a quick and pleasing result. For other groups of users this closed design is more limiting. Independent creators who want to stylise their own images of-ten want some control over how strongly a style is applied. Designers and artists who wish to incorporate style transfer into a larger body of work similarly benefit from finer control over the visual outcome. Computing students studying machine learning or computer vision are better served by a system that lets them see how a design choice, such as the network used or the layers selected, changes a real output, rather than by a system that conceals those choices entirely. Gatys, Ecker, and Bethge (2016) themselves introduce a weighting parameter to govern the balance between content and style, indicating that even in the original for-

mulation of the method this balance was understood to matter and to warrant control, a point that closed consumer applications tend not to expose to their users. Research in human-computer interaction has similarly found that users working with generative and creative tools report greater engagement and more deliberate decision-making when the parameters of those tools are exposed rather than fixed [4].

The project addresses a narrower and more achievable version of this problem. Existing accessible tools conceal the decisions that determine the quality and character of the output. This project instead builds a system in which those decisions, and their effects, are made visible and testable.

1.3 Aims and Research Questions

The project aims to design, implement and evaluate an NST system that reveals how the underlying network, its key settings, and its optimisation method affect the image it produces, in a way that can be followed and repeated by a non-technical user. Three research questions guide the work.

Research Question 1: How the choice of convolutional backbone, comparing VGG16 against VGG19, together with the layers selected to represent content and style, affects the quality of the generated image.

Research Question 2: How sensitive the result is to the balance struck between style and content in the loss function, and what patterns emerge as that balance is varied.

Research Question 3: How two optimisation algorithms, stochastic gradient descent and Adam, compare when used to drive the style transfer process, with attention to convergence speed, runtime, and the quality of the resulting image.

Together these questions move the investigation beyond a single working pipeline toward a broader account of how architectural, hyperparameter and optimisation choices shape the outcome of neural style transfer.

1.4 Scope and Limitations

The project centres on the optimisation-based method of Gatys, Ecker, and Bethge (2016), using pre-trained VGG16 and VGG19 networks as fixed feature extractors rather than training new networks from scratch. The comparison work carried out spans three areas: the choice of backbone and layer selection, the weighting between style and content loss, and the choice of optimiser, evaluated using both stochastic gradient descent and Adam [5]. Runtime is recorded as an outcome of these comparisons rather than treated as a target to be minimised, since the priority of the project is understanding the effect of each choice rather than achieving the fastest possible result. The project runs on Google Colab Pro for GPU access, without which the iterative optimisation involved in each run would be too slow to complete within the time available for the project.

2 Literature Review (1984 words)

2.1 Introduction

Neural Style Transfer brings together computer vision, deep learning and computational creativity. This review covers four areas relevant to the project. It begins with the optimisation-based method that first made NST possible and the network architecture it relies on, then turns to later work that made style transfer faster and more general, before examining how the choice of optimisation algorithm affects an iterative image synthesis process. It closes with a review of how stylised images are evaluated in the literature. Each area is considered for what it contributes to the project and where it falls short, and together they point to the gap the project sets out to address through its three research questions.

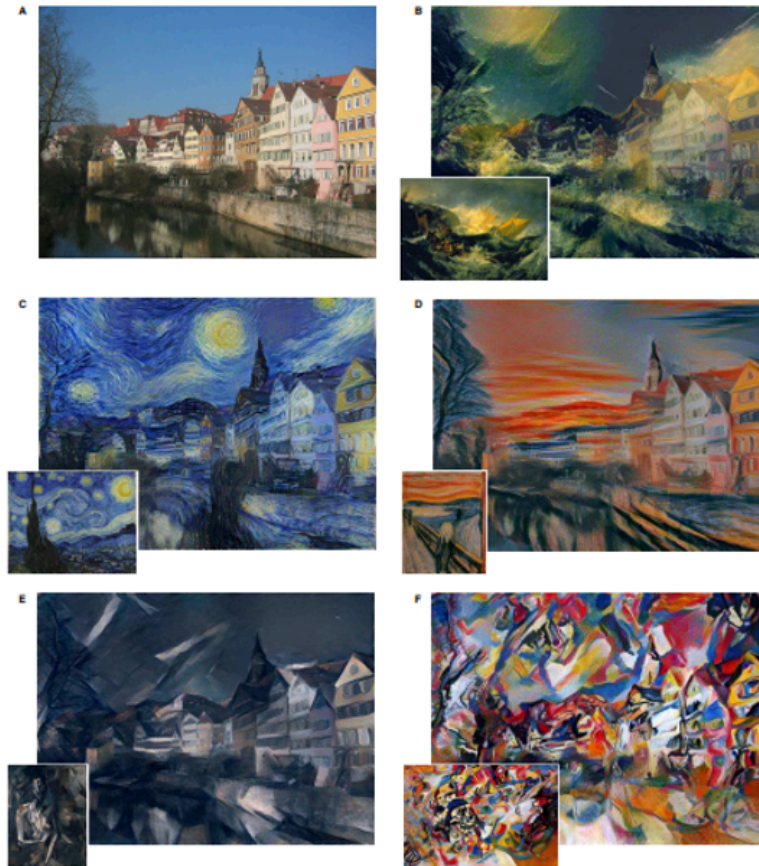


Figure 2: A single photograph rendered in the styles of several artworks using neural style transfer. Reproduced from Gatys, Ecker and Bethge (2016) for the purposes of criticism and review.

2.2 Optimisation-Based NST Foundations

The field began with Gatys, Ecker, and Bethge (2016), who showed that a convolutional neural network trained to recognise objects also learns, as a by-product, to separate an image's subject matter from its appearance. Their central observation is that the deeper layers of such a network describe high-level content, meaning which objects are present and roughly where, while appearance can be summarised by the correlations between feature channels within a layer. These correlations are written as a Gram matrix, which retains texture and colour statis-

tics but discards spatial position. A new image is produced by adjusting its pixels step by step until they match both the content of a photograph and the style of an artwork, with a weighting between the two losses controlling how strongly the style is imposed. This process is illustrated in Figure 3. The method requires no new training data or per-style network, since any pre-trained classification network can be repurposed as a feature extractor for any content and style pairing.

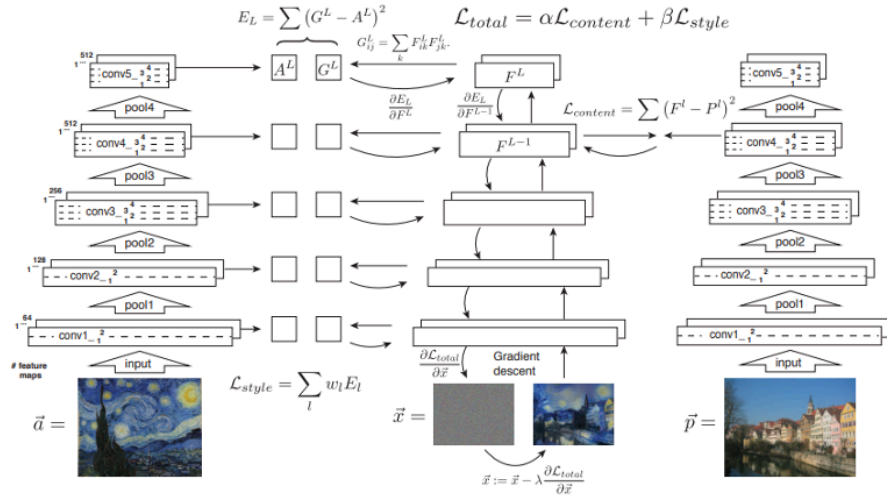


Figure 3: The neural style transfer algorithm. Content and style features are extracted from a pre-trained VGG network and recombined into the generated image. Reproduced from Gatys, Ecker and Bethge (2016) for the purposes of criticism and review.

The network used for this purpose is VGG, introduced by Simonyan and Zisserman (2014), a deep architecture built from stacked small convolutional filters and made available in several depths, including 16-layer and 19-layer configurations. Gatys, Ecker, and Bethge (2016) used the 19-layer version, and did not report results for the 16-layer variant, leaving open the question of whether a shallower network with a smaller receptive field produces comparably useful content and style representations. Simonyan and Zisserman (2014) designed their architecture for image classification, not for perceptual style representation, so neither paper

offers a principled account of why particular layers are well suited to separating content from style. Gatys, Ecker, and Bethge (2016) select their layers empirically rather than deriving them from theory, and note in their own discussion that both the layers used and the weighting applied to the two losses have a strong effect on the resulting image. This sensitivity, acknowledged but not systematically explored in the original work, is what Research Question 1 and Research Question 2 of this project examine directly, the first by comparing backbones and layer sets, the second by varying the style and content weighting.

The main weakness of the method is cost. Each image requires hundreds or thousands of optimisation steps, so producing a single result can take a considerable amount of time even on a GPU, which rules the method out of real-time use. This weakness, together with the one-style-at-a-time nature of the approach, is what motivated the next generation of NST research, considered in the following section.

2.3 Toward Speed and Generality

Johnson, Alahi, and Fei-Fei (2016) addressed the speed problem by changing the approach entirely. Rather than optimising each image from scratch, they trained a feed-forward network that produces a stylised image in a single pass, using a perceptual loss defined over the same VGG features that Gatys, Ecker, and Bethge (2016) use directly. Once trained, the network stylises a new photograph in milliseconds rather than minutes, which makes style transfer usable in interactive or mobile settings. Its limitation is that each trained network reproduces only the one style it was trained on, so applying a different artwork means training an entirely new network, an expensive step that also removes any opportunity for a user to adjust the loss weighting after the fact.

Huang and Belongie (2017) addressed the one-style limitation with Adaptive Instance Normalisation, which matches the mean and variance of the content image's features to those of the style image inside a single trained network. This

allows one model to handle any style in real time, combining the speed of Johnsson, Alahi, and Fei-Fei (2016) with the generality of Gatys, Ecker, and Bethge (2016). The trade-off is that matching only two summary statistics per channel is a coarser representation of style than the full Gram matrix, and the resulting images can reproduce a style loosely or let the content image's own colours bleed through. Li et al. (2017) pursued a related goal using whitening and colouring feature transforms, achieving similar flexibility through a different mechanism. Together, these fast and general methods are what made consumer applications such as Prisma possible, and are also what standardized the trade-off, discussed in Chapter 1, between an accessible tool and one that exposes its internal decisions to the user.

For this project these methods are background rather than the implementation target. Each of them hides the network, the layers and the loss weighting inside a trained model, which is precisely the opacity that Chapter 1 identifies as a limitation for independent creators, designers and computing students who want to see and adjust how a result is produced. This project instead uses the slower optimisation method of Gatys, Ecker, and Bethge (2016), because it keeps the backbone, the layer selection and the loss weighting visible and adjustable at the point of use. Choosing to keep the process visible, however, still leaves open a further question that none of the papers considered so far addresses directly, namely how the optimisation algorithm driving the iterative process itself affects the outcome.

2.4 Optimisation Algorithms in Iterative Image Synthesis

Gatys, Ecker, and Bethge (2016) originally optimised the generated image using L-BFGS, a quasi-Newton method well suited to the small number of variables involved in early implementations. Later tutorial implementations, including the one presented by Chollet (2018) that this project uses as a code reference, instead use gradient-based optimisers such as stochastic gradient descent, since quasi-Newton

methods are less readily supported in frameworks such as TensorFlow and Keras. Stochastic gradient descent with a decaying learning rate is a long-established and simple approach, updating the image directly by its loss gradient and gradually reducing the step size to allow the optimization to settle. Its simplicity is also its limitation, since performance depends on a learning rate schedule that must be chosen carefully for the specific loss surface being optimised.

Kingma and Ba (2015) introduced Adam as an alternative that adapts its learning rate for each parameter individually, using running estimates of the first and second moments of the gradient. This adaptive behaviour generally makes Adam more robust to the initial choice of learning rate and is one reason it has become a common default for training neural network weights. Kingma and Ba (2015) evaluate Adam on conventional supervised learning tasks, where the quantity being updated is a large set of network weights shaped by a training dataset. Iterative image synthesis is a different setting, since the quantity being updated is the pixel content of a single image, a comparatively low-dimensional and un-structured search that is not represented in the original Adam paper or in the broader optimization literature reviewed here. Whether an optimiser designed and evaluated for weight training behaves comparably when applied to pixel-level optimisation is therefore an open question rather than an established finding, and is what Research Question 3 of this project sets out to examine by comparing stochastic gradient descent and Adam under the same style transfer configuration.

2.5 Evaluating Neural Style Transfer

Evaluating NST is difficult because the goal is subjective. There is no single correct output, only ones that look more or less convincing to a given viewer. Jing et al. (2019) survey the field and group methods into image-optimisation approaches such as Gatys, Ecker, and Bethge (2016) and model-optimisation approaches such as Johnson, Alahi, and Fei-Fei (2016) and Huang and Belongie (2017). Their review also covers how quality is measured, and finds two broad approaches: qual-

itative judgement by human observers and quantitative metrics. Among the latter, the content and style losses defined by Gatys, Ecker, and Bethge (2016) are some-times used directly as a proxy for quality, and the structural similarity index of Wang et al. (2004), commonly abbreviated to SSIM, is used elsewhere in image processing to measure how well the structure of a reference image is preserved. Jing et al. (2019) note that the field has reached no agreed quantitative standard for style transfer specifically, a limitation that follows directly from the subjective nature of style and that makes any single metric an incomplete measure of quality on its own.

The project responds to this gap by using more than one measure rather than searching for a single definitive one. Loss values are recorded because they are objective and directly tied to what the optimisation is minimising. SSIM is recorded because it gives an independent, content-focused measure that does not depend on the style loss formulation. A viewer survey was planned to capture the qualitative judgement that loss values and SSIM cannot, but was not conducted within the time available for the project; the qualitative assessment in Chapter 5 is therefore based on direct visual inspection of the output images. None of these measures is treated as sufficient on its own, and their combination, along with wall-clock timing for the optimiser comparison, forms the basis of the evaluation carried out in Chapter 5.

2.6 Comparative Summary

Table 1 draws together the approaches discussed in this chapter, alongside the two optimisers compared within this project. The first three columns reflect prop-erties reported by each method’s own authors. The user control and transparency column is this review’s own comparison rather than a figure the papers report directly.¹ The entry for consumer applications reflects observed behaviour rather

¹The entry for consumer applications reflects observed behaviour rather than published detail, since applications such as Prisma do not disclose their internal configuration.

than published details.

Table 1: Comparison of neural style transfer approaches by generality, speed, training requirement, transparency of control, and quality measure.

Approach	Arbitrary style	Speed	Per-style training	User control
Gatys et al. [3]	Yes	Slow	No	High
Johnson et al. [7]	No	Real-time	Yes	Low
Huang & Belongie [8]	Yes	Real-time	No	Low
Consumer apps	Fixed	Real-time	N/A	None
This project, SGD	Yes	Slow	No	High
This project, Adam	Yes	Slow	No	High

2.7 Discussion and Identified Gap

Taken together, the literature reviewed here shows a consistent pattern. The optimisation-based method of Gatys, Ecker, and Bethge (2016) offers flexibility and transparency, since it works with any content and style pairing and exposes every setting involved, but it is slow and sensitive to choices of layer selection, loss weighting and, as the discussion of optimisation algorithms in Section 2.4 suggests, possibly the optimiser itself, that the original work and its successors do not explore systematically. The faster methods that followed, from Johnson, Alahi, and Fei-Fei (2016) to Huang and Belongie (2017) and Li et al. (2017), resolve the speed problem but do so by moving those same choices inside a trained model, where they are no longer visible or adjustable. Evaluation practice, as reviewed by Jing et al. (2019), remains similarly unsettled, relying on a mixture of objective and subjective measures because no consensus exists on a single adequate one.

The gap this leaves is that a user of any existing tool has little basis for understanding how the choices the research shows to be consequential interact to produce a given result. This project addresses that gap directly by keeping the transparent formulation of Gatys, Ecker, and Bethge (2016) as its implementation, and by structuring its investigation around the three points of uncertainty iden-

tified in this chapter: backbone and layer selection in Research Question 1, style and content weighting in Research Question 2, and optimiser choice in Research Question 3, evaluated using the combined quantitative and qualitative approach set out in Section 2.5.

3 Project Design (1849 words)

3.1 Project Overview

The system takes a content image and a style image and returns a stylised picture using the optimisation method of Gatys, Ecker and Bethge (2016), with a pre-trained VGG network held fixed as a feature extractor while the pixels of the output image are adjusted by gradient descent. The pipeline is wrapped in an experimental harness that varies the backbone, layer selection, loss weighting and optimiser in turn, logging the resulting images and loss values so results can be compared on equal terms. This structure follows directly from the three research questions introduced in Chapter 1, each of which requires a different dimension of the pipeline to be treated as a controlled variable rather than a fixed setting.

3.2 Domain and Users

The project sits in computer vision and computational creativity, specifically the generation of stylised images using convolutional neural networks. It applies an established technique rather than inventing a new one, and the contribution lies in who can use it and how much of the process they can see and control, a distinction motivated by the evidence discussed in Section 1.2.

The three user groups identified in Section 1.2 each have a specific need the design responds to. Independent creators are served by exposing the style and content weighting as an adjustable setting, the subject of Research Question 2. Designers and artists are served by exposing the layer configuration and record-

ing every parameter behind a result so it can be reproduced exactly. Computing students are served by the comparison harness itself, since the loss values, structural similarity scores and timings recorded per configuration give a quantitative account of what changed and why, addressed across Research Questions 1 to 3.

3.3 Justification of Design Choices

The project uses the optimisation method of Gatys, Ecker and Bethge (2016) rather than a feed-forward alternative such as Johnson, Alahi and Fei-Fei (2016) or Huang and Belongie (2017), discussed in Section 2.3. Both alternatives are considerably faster, but a feed-forward network hides its backbone, layer selection and loss weighting inside trained weights, and Johnson, Alahi and Fei-Fei (2016) additionally requires a separate network per style, neither of which suits the three user groups above, all of whom need these settings to be visible or adjustable. The slower method keeps every setting exposed at the point of use, which is why its runtime cost is accepted.

Both VGG16 and VGG19, introduced by Simonyan and Zisserman (2014), are tested because Gatys, Ecker and Bethge (2016) report results only for the 19-layer network, leaving open whether a shallower backbone produces comparable results, the question Research Question 1 addresses. The content layer is fixed at block5_conv2 for both backbones so the network and layer set are the only variables changing between runs. Three style layer configurations are tested for the same reason: a default set spanning block1_conv1 through block5_conv1, a shallow-only set using block1_conv1 and block2_conv1, and a deep-only set using block4_conv1 and block5_conv1, so the effect of favouring early, texture-level features against late, structure-level features can be observed rather than assumed.

Stochastic gradient descent with exponential learning rate decay is used because it is the approach taken in the Chollet (2018) implementation used as a code reference, and remains a standard default for this kind of iterative optimisa-

tion. Adam, introduced by Kingma and Ba (2015), is tested alongside it because its adaptive, per-parameter learning rate is a distinct strategy whose behaviour in pixel-space image synthesis, rather than conventional weight training, is not addressed in the original literature, the gap Research Question 3 investigates.

The project runs on Google Colab Pro with a Tesla T4 GPU because the comparison harness in Section 3.6 involves twenty separate runs of two thousand iterations each, too much computation for a practical CPU timeframe. Colab Pro was chosen over the free tier for its longer, more reliable GPU session limits, which the full sweep requires.

3.4 System Architecture

The pipeline runs in five stages, shown in Figure 4. The figure depicts the content and style images entering as inputs, both passing through the frozen VGG feature extractor, which branches into a content loss and a style loss computed from Gram matrices, alongside a separate total variation loss computed on the generated image. The three losses combine into a weighted total loss, driving an optimisation loop that updates the generated image until the final stylised output is produced.

In the first stage, the content and style images are loaded and resized to a height of 400 pixels while preserving aspect ratio, then mean-centred by subtracting the per-channel ImageNet means expected by VGG, with a batch dimension added before either image enters the network. In the second stage, a VGG16 or VGG19 network pre-trained on ImageNet is loaded with its classification layers removed, and its intermediate layer outputs are exposed as a named dictionary so a single forward pass yields content features from block5_conv2 and style features from whichever layer set is under test, for both reference images and the generated image at each step.

In the third stage, the content loss is computed as the L2 distance between the generated image's features and the content image's features at block5_conv2. The

quantity being optimised. The gradient of the total loss with respect to the image is computed inside a function compiled with `@tf.function` for GPU efficiency, and either SGD or Adam applies the resulting update to the image pixels. SGD uses an initial learning rate of 100.0 with exponential decay applied every 100 steps at a rate of 0.96, and Adam uses a learning rate of 10.0, calibrated separately against real images before the Research Question 3 comparison. Each configuration runs for 2000 iterations, with a checkpoint saved every 200.

In the fifth stage, the final image is deprocessed by adding back the ImageNet channel means, converting from BGR to RGB channel order, and clipping pixel values to the valid 0 to 255 range. The resulting image, its final loss components, its structural similarity score against the content image, and the wall clock time taken are written to `results_log.csv`, recording every run in a common format.

3.5 Technologies and Methods

The project is implemented in Python 3.13, in Jupyter notebook format, on Google Colab Pro with a Tesla T4 GPU, for the reasons given in Section 3.3. Deep learning is handled by TensorFlow 2.20 with the Keras applications module, which provides the pre-trained VGG16 and VGG19 networks used as feature ex-tractors without requiring any training data of the project’s own. NumPy handles array operations, Pillow loads and saves image files, and Matplotlib displays and saves comparison figures. Scikit-image 0.25.2 provides the structural similarity implementation used for SSIM in Section 3.8, following Wang et al. (2004), as an established implementation rather than one written specifically for this project.

3.6 Experiment Design

The three research questions are structured as separate sweeps, each varying one factor while the rest are held constant, so any difference in output can be attributed to the factor under test. Research Question 1 covers six configurations,

crossing both backbones, VGG16 and VGG19, with all three layer configurations, while loss weights, optimiser and iteration count stay fixed. Research Question 2 covers six configurations, sweeping the style weight across four values from 1×10^{-4} to 1×10^{-1} at a fixed total variation weight, then repeating two of those style weights at alternative total variation weights of 1×10^{-9} and 1×10^{-7} , with the backbone fixed at VGG19 and the default layer set. Research Question 3 covers two configurations, SGD against Adam, with every other parameter carried forward from the Research Question 1 and 2 configurations. A separate gallery of six runs, crossing three content images with two style images, applies the pipeline under a fixed configuration to demonstrate the system across subject matter rather than to test a variable.

3.7 Risks and Contingencies

The risk of insufficient compute for twenty full-length runs was managed using Google Colab Pro with a Tesla T4 GPU, and by saving a checkpoint every 200 iterations so an interrupted session loses at most a small portion of a run. Where a backbone or layer configuration produces a result that cannot serve as a valid comparison point, the remaining configurations within the same research question retain a basis for comparison. Loss weight sensitivity emerged as a further risk during implementation and is reported in Section 4.5.

3.8 Testing and Evaluation Plan

Testing was carried out at three levels before the experimental runs were conducted. At component level, the loss functions and Gram matrix computation were validated against synthetic tensors of known shape to confirm the channel normalisation and loss arithmetic behaved as intended, catching the channel-count error in Section 3.4 before it affected a real run. At integration level, the full pipeline was run as an end-to-end test on small synthetic images at 32-pixel resolution to

confirm every stage, from preprocessing through to logging, completed without error before committing GPU time to the 400-pixel, 2000-iteration configurations. At system level, the experiment runner checks `results_log.csv` for completed experiment records and skips them, so an interrupted session can resume without repeating finished runs.

Evaluation combines quantitative and qualitative measures, following Section 2.5. Quantitatively, the final content loss, style loss, total variation loss and total loss are recorded for every run in `results_log.csv`, allowing the contribution of each component to be inspected rather than only the combined figure. The structural similarity index, following Wang et al. (2004), is computed between the content image and the generated output, with values closer to 1.0 indicating greater preservation of content structure. Wall clock time is recorded for every run, used specifically to compare SGD against Adam in the Research Question 3 analysis. Qualitatively, a viewer survey was planned to present participants with pairs of stylised images and ask them to rate each across three aspects: how strongly the reference painting’s style is present, how clearly the original content subject remains recognisable, and an overall preference between the paired outputs. The survey was not conducted within the time available for the project. The qualitative assessment carried out in Chapter 5 is therefore based on direct visual inspection of the output images.

4 Implementation (906 words)

4.1 Environment and Setup

The notebook runs on TensorFlow 2.20.0 with Python 3.13, Keras 3.13.2, NumPy 2.1.3, Pillow 11.3.0 and scikit-image 0.25.2 for the SSIM computation described in Section 3.8. A `write_requirements_txt` function captures the exact installed versions via `pip freeze` at the end of each session for reproducibility.

Google Drive is mounted at startup and PROJECT ROOT is set to a folder on Drive, with a fixed directory structure created beneath it for content and style images, per-research-question results, checkpoints, figures and survey material.

results log.csv is initialised with a fixed column schema covering run id, timestamp, rq, backbone, layer set, optimizer name, content image, style image, content weight, style weight, tv weight, iterations, final content loss, final style loss, final tv loss, final total loss, wall clock seconds, ssim_content output and output path. Initialisation is idempotent, creating the file only if it does not already exist and leaving it untouched otherwise.

4.2 Core Functions

Image handling is provided by preprocess_image, deprocess_image and compute_output_dims. preprocess_image takes the VGG preprocessing function as an explicit argument so the same function serves both VGG16 and VGG19 without duplication.

build_feature_extractor loads the requested backbone with pretrained ImageNet weights and its classification head removed, exposing every layer's output as a named dictionary. A companion function, validate_layers_exist, confirms that every requested layer name is present in that dictionary before optimisation begins, catching configuration errors immediately rather than partway through a long run.

The loss functions gram_matrix, content_loss, style_loss and total_variation_loss implement the formulation described in Chapter 2. The style_loss contains one cor-

rection from the implementation presented by Chollet (2018), which normalised the

Gram matrix by a hardcoded value of three channels rather than the true channel count of each layer. The corrected version uses `tf.cast(tf.shape(style)[-1], tf.float32)`,

which matters for Research Question 1 since the style layers range from 64 to 512 channels, and the original normalisation would have scaled each layer's contribution differently depending on its depth rather than its actual role. compute_loss returns all

- four loss components rather than only the combined

scalar so each can be logged and plotted separately. make loss and grads fn wraps this in a `tf.GradientTape` compiled with `@tf.function` for GPU efficiency, with a fresh instance created per run to ensure each configuration uses its own correct layer and weight values.

4.3 Experiment Runner

`run single experiment` is driven entirely by a configuration dictionary. It builds the feature extractor and optimiser, initialises the generated image as a copy of the content image, runs the gradient descent loop and returns a result dictionary shaped as one results log.csv row.

`run sweep` passes a list of configurations through `run single experiment`, skipping any whose run identifier already appears in results log.csv. Failed configurations are caught and written to failed runs.csv without stopping the remaining runs. Each run identifier encodes all varying parameters in one string, ensuring every configuration produces a unique and reproducible identifier. A checkpoint image and loss history are saved every 200 iterations, providing the convergence data.

4.4 Weight Recalibration

The initial loss weights from the implementation presented by Chollet (2018) were style weight=1e-6 and total variation weight=1e-6. At these values the total variation loss dominated while the style loss remained close to zero, producing output images with almost no visible style transfer. The weights were calibrated empirically to style weight=1e-3 and total variation weight=1e-8, the values used throughout all RQ1 to RQ3 and gallery runs.

4.5 Adam Learning Rate Calibration

Adam’s learning rate cannot share SGD’s initial value of 100.0 because the two optimisers scale their updates differently, as discussed in Section 2.4. A `calibrate_adam_lr` function ran 50-iteration trials at four candidate values, 0.01, 0.1, 1.0 and 10.0, against the real content and style images at full resolution. The value `lr=10.0` produced the lowest finite loss and was adopted for the Research Question 3 sweep.

4.6 RQ1 Implementation

`build_rq1_configs` generates six configurations by pairing VGG16 and VGG19 with each of the three layer sets, `default`, `shallow only` and `deep only`, with all other parameters fixed. Backbone and layer set are the only dimensions that vary across the six runs.

4.7 RQ2 Implementation

`build_rq2_configs` sweeps `style_weight` across four values, 1×10^{-4} , 1×10^{-3} , 1×10^{-2} and 1×10^{-1} , at a fixed total variation weight, then sweeps `total_variation_weight` across 1×10^{-9} and 1×10^{-7} at the baseline style weight, for six configurations in total. The backbone is fixed at VGG19 with the default layer set.

4.8 RQ3 Implementation

`build_rq3_configs` produces two configurations varying only the optimiser, SGD and Adam, with all other parameters fixed at the VGG19 default configuration. L-BFGS, used in the original Gatys, Ecker, and Bethge (2016) implementation, was considered but excluded because its scipy-based implementation requires converting between GPU tensors and NumPy arrays on every iteration,

which is incompatible with the TensorFlow gradient computation used throughout this project.

4.9 Gallery Implementation

`build_gallery_configs` pairs three content images with two style images for six runs under the configuration carried forward from RQ1, RQ2 and RQ3. The content images cover an interior, a portrait and a mountain landscape. The style images are an abstract geometric composition and an Impressionist painting, along-side the *Starry Night* image used as the style reference throughout the earlier sweeps. The gallery demonstrates that the pipeline generalises across different content and style pairings rather than testing a controlled variable.

5 Evaluation (1169 words)

5.1 Overview of Evaluation Approach

Evaluation follows the plan set out in Section 3.8, combining quantitative measures from `results_log.csv` with direct visual inspection of the output images in place of the planned viewer survey.

5.2 RQ1: Backbone and Layer Set Comparison

The primary division in Table 2 falls between layer sets rather than between backbones. The deep only configurations produce a distinct pattern from the rest, with markedly lower style loss and substantially higher SSIM scores. This outcome is consistent with the point made in Chapter 2 that deeper layers encode structural content rather than fine texture: when style is represented only through these layers, the Gram matrices capture edges and shapes rather than the colour and brushstroke statistics the method is intended to transfer. The low style loss in these runs does not indicate successful stylisation; it indicates that the optimisation

Table 2: RQ1 results for all six backbone and layer set configurations. Sin-gapore content image and Starry Night style image, style_weight= 1×10^{-3} , tv_weight= 1×10^{-8} , 2000 iterations.

Backbone	Layer set	Content loss	Style loss	TV loss	Seconds	SSIM
VGG16	default	3.79	288.59	107.15	427.8	0.635
VGG16	shallow_only	3.07	336.78	108.21	396.8	0.674
VGG16	deep_only	0.65	30.83	33.11	361.3	0.985
VGG19	default	12.15	182.73	109.27	500.7	0.616
VGG19	shallow_only	5.86	274.86	110.15	469.9	0.660
VGG19	deep_only	14.22	47.51	64.17	436.7	0.700

found it straightforward to match structural statistics it should not have been targeting.

At a fixed layer set, the two backbones behave similarly. VGG19 consistently reaches a lower style loss than VGG16 at the same configuration, though since Gram matrix magnitudes are not necessarily on the same scale across architectures this difference is reported as an observation rather than a conclusion about which backbone is more capable. VGG16 runs faster at every layer set, which is expected given its smaller depth. The practical implication is that choosing between the two backbones has a smaller effect on the character of the output than choosing between layer sets.

Figure 5 illustrates these patterns visually. The default and shallow only out-puts across both backbones show the colour shift and textural qualities associated with the Starry Night style image, while the deep only outputs retain greater similarity to the original photograph regardless of backbone. The degree of stylistic transformation varies substantially by layer set across all six configurations.

5.3 RQ2: Style and Content Loss Weighting

The style weight experiment produces a continuous progression, as the penalty on style mismatch increases. The optimisation prioritises matching the painting’s

RQ1: VGG16 vs VGG19 x Layer Sets
Content: Singapore Skyline | Style: Starry Night
style_weight=1e-3, tv_weight=1e-8, iterations=2000

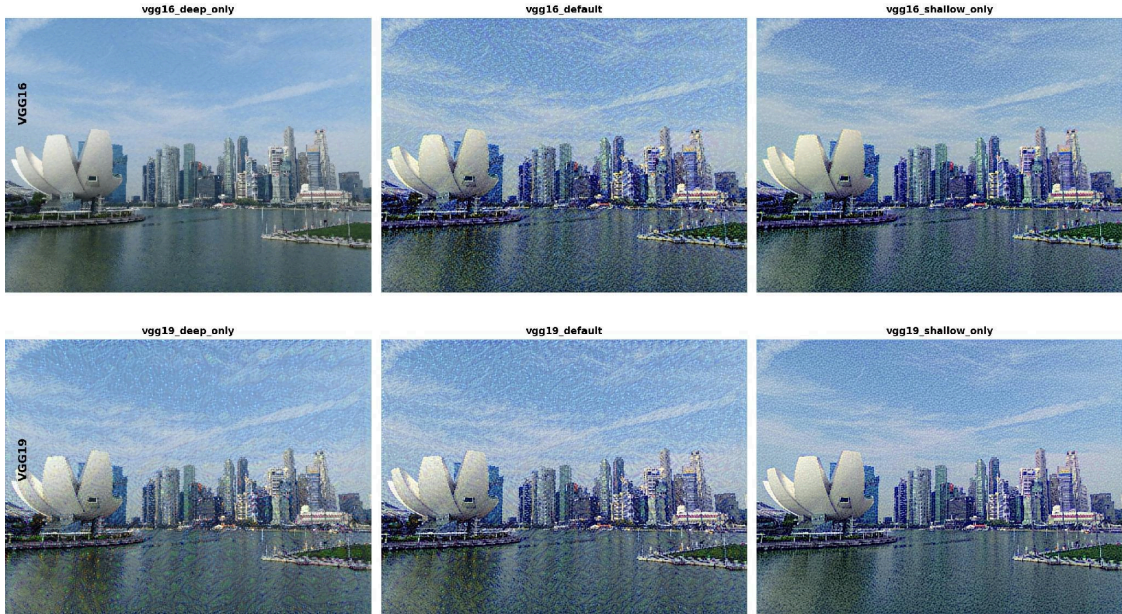


Figure 5: RQ1 visual comparison: all six backbone and layer set configurations on the Singapore content image and Starry Night style image. Each panel is labelled with its backbone, layer set, final style loss and SSIM.

texture and colour statistics over preserving the photograph’s structure, and SSIM falls accordingly. The direction of this trade-off is consistent with expectations, but the range of the effect is notable: the four style weight values tested correspond to outputs spanning from one in which photographic structure is largely retained to one in which the scene’s palette and brushwork are predominant.

The total variation weight sweep shows a different pattern. The effect on the visual output is subtle compared to the style weight sweep, but the TV loss value itself grows disproportionately at the highest TV weight tested, echoing the same dominance behaviour described in Section 4.4 at a smaller scale. Smoothness and stylisation pull against each other, but the TV weight is a less direct lever on artistic character than the style weight.

Figure 6 illustrates the style weight progression. The shift from a content-dominant to a style-dominant output is gradual across the four weight

values, with

Table 3: RQ2 results, VGG19/default, Singapore content image and Starry Night style image, 2000 iterations.

Style weight	TV weight	Content loss	Style loss	TV loss	SSIM
1×10^{-4}	1×10^{-8}	7.07	173.05	62.29	0.821
1×10^{-3}	1×10^{-8}	12.14	182.74	109.27	0.616
1×10^{-2}	1×10^{-8}	17.70	470.39	108.41	0.604
1×10^{-1}	1×10^{-8}	23.71	444.25	108.87	0.474
1×10^{-3}	1×10^{-9}	12.15	182.68	12.52	0.624
1×10^{-3}	1×10^{-7}	12.78	252.68	562.03	0.556

each successive setting introducing more of the style image’s colour and texture while the underlying skyline structure remains identifiable throughout the range tested.

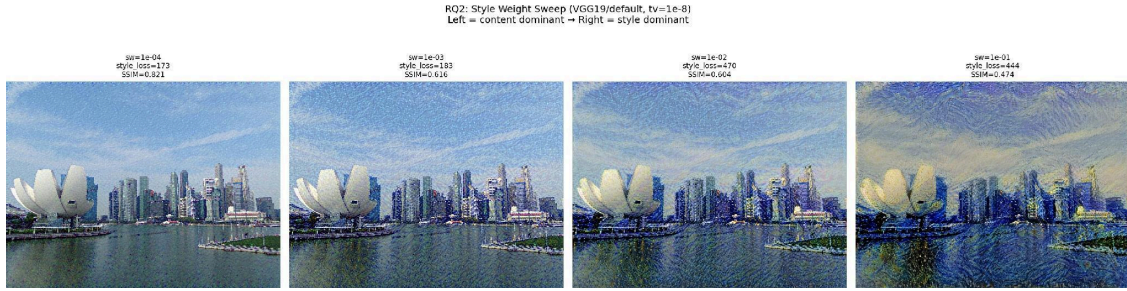


Figure 6: RQ2 style weight sweep: outputs at four style weight values on the Singapore and Starry Night pair. Left to right: 1×10^{-4} , 1×10^{-3} , 1×10^{-2} , 1×10^{-1} .

5.4 RQ3: Optimiser Comparison

Wall-clock time is identical to within one second across a 2000-iteration run, so speed offers no practical basis for choosing between the two optimisers under this configuration.

What differentiates them is where each converges. Adam substantially reduces the style loss, reaching a numerically much tighter match to the Starry Night statistics, but at a severe cost to content structure as measured by SSIM. SGD

Table 4: RQ3 results, VGG19/default, Singapore content image and Starry Night style image, style_weight= 1×10^{-3} , tv_weight= 1×10^{-8} , 2000 iterations.

Optimiser	Content loss	Style loss	TV loss	Total loss	Seconds	SSIM
SGD	12.15	182.74	109.27	304.15	501.5	0.616
Adam	5.05	14.19	52.84	72.07	501.2	0.104

distributes the loss more evenly across all three components and retains far more of the original photograph. Under identical settings, the two optimisers converge to qualitatively distinct outputs in the configuration tested.

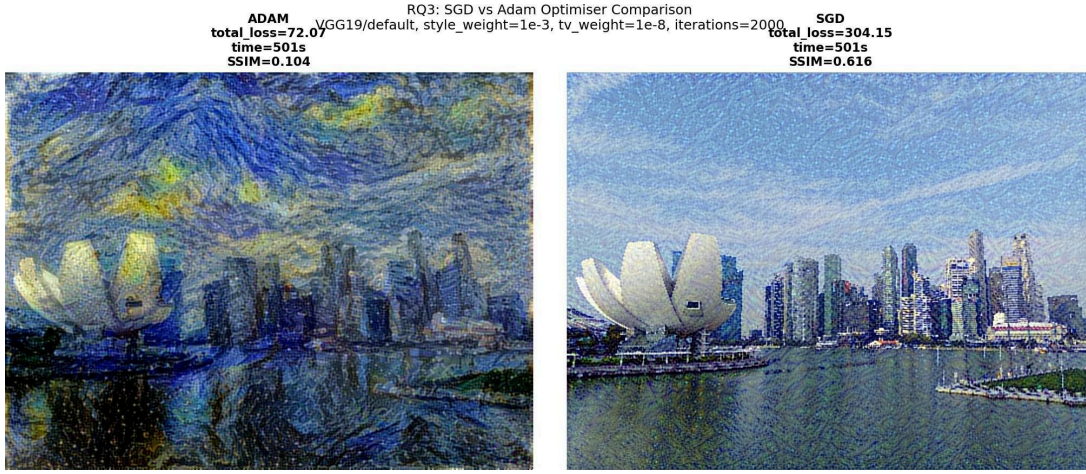


Figure 7: RQ3 optimiser comparison: SGD (left) and Adam (right) on the Singapore and Starry Night pair at identical settings.

Figure 7 illustrates these differences. Adam’s output shows stronger application of the style image’s palette and brushwork; SGD’s output retains greater photographic detail with the stylisation present to a lesser degree. Neither outcome is objectively preferable. In the configuration tested, the choice between optimisers functions as a creative parameter: Adam is suited to use cases requiring stronger artistic transformation, while SGD is suited to those where content fidelity is the priority.

5.5 Gallery Results

The gallery demonstrates that the pipeline generalises across subject matter. Applying the same configuration to an interior photograph, a portrait, and a mountain landscape produces recognisably stylised outputs in every case, confirming that the method is not specific to the urban skyline used across RQ1 to RQ3.

The two style images produce consistent behaviour across all three content images. The geometric style image imposes a hard-edged, crackled texture that interacts differently depending on the content. Applied to the portrait, where fine facial detail is present, the dense linear overlay reduces subject legibility to a greater degree than in the other pairings. The impressionist style image produces a softer, more diffuse texture that preserves legibility across all three content images; the pairing with the mountain landscape shows the closest correspondence between the broad tonal areas of that image and the diffuse texture the impressionist style produces. Content and style compatibility is a relevant factor, as Jing et al. (2019) observe, and the gallery provides a practical illustration of how it varies across pairings.

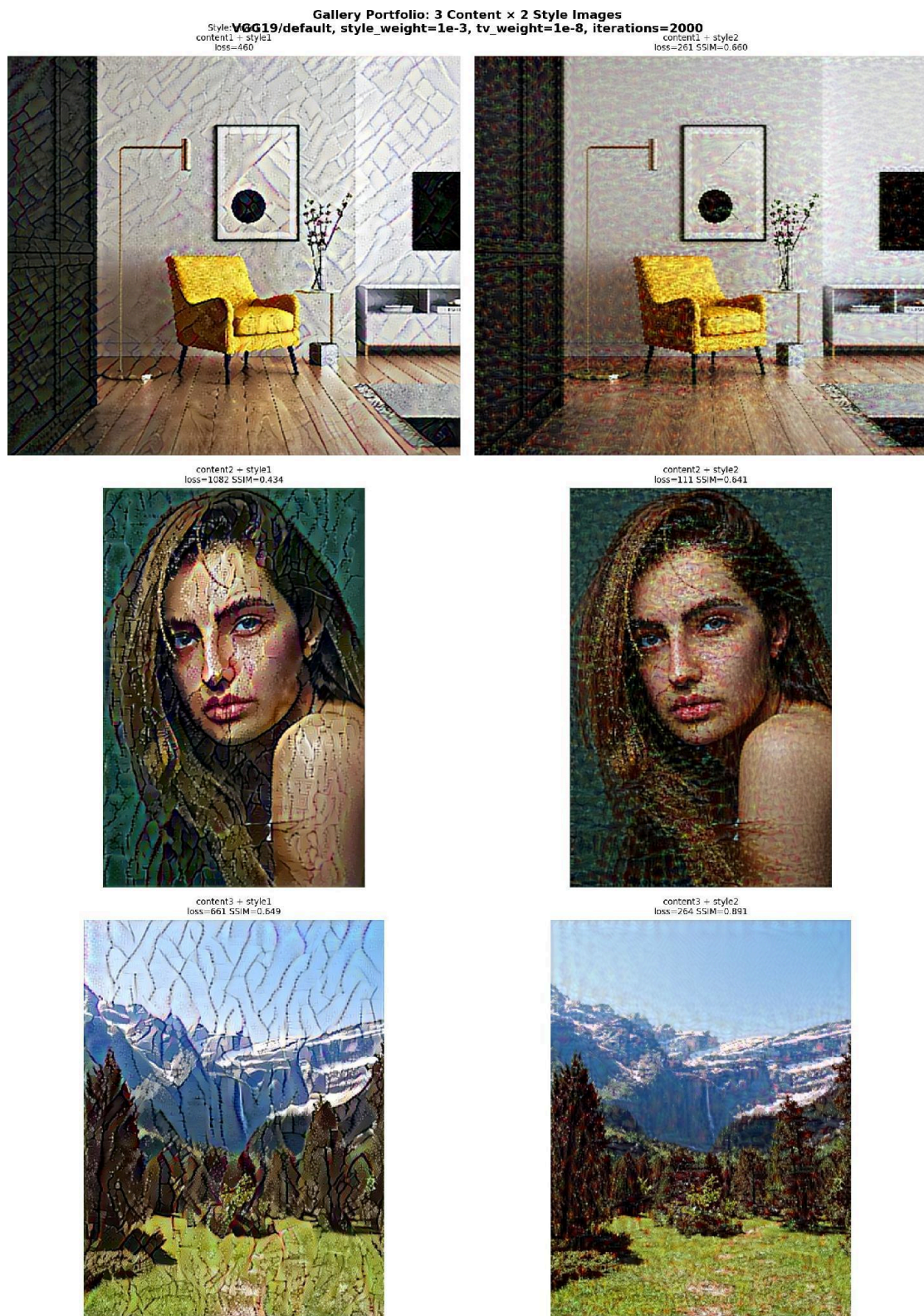


Figure 8: Gallery results: three content images (interior, portrait, mountain land-28 scape) paired with two style images (Kandinsky-like geometric, Monet-like impressionist). Six runs at VGG19/default configuration.

5.6 Critical Evaluation

A consistent pattern runs across all four sets of results: stronger stylisation and better content preservation pull against each other, regardless of which parameter drives the trade-off. Layer set, loss weighting, optimiser choice and style image character each produce the same directional effect, and the quantitative metrics and visual inspection agree throughout. This inverse relationship is not itself surprising, but observing it consistently across independent variables strengthens confidence that it reflects a genuine property of the method rather than an artefact of any single configuration.

Among the findings, layer set selection has a more pronounced effect on out-put character than backbone choice. The choice between VGG16 and VGG19 produces modest differences that do not resolve clearly in favour of either architecture; the choice of which layers represent style produces qualitatively distinct outputs. This has a direct practical implication for anyone implementing the Gatys method: default layer configurations taken from reference implementations may not transfer to different images or frameworks without adjustment, a point the weight recalibration episode in Section 4.4 also illustrates.

The project answered all three research questions using real experimental data, and the RQ3 finding that two widely used optimisers reach qualitatively distinct solutions under identical settings is reported here as an empirical observation on this image pair and configuration. Several limitations qualify the findings: all backbone and layer set comparisons rest on a single image pair, no viewer survey was conducted, SSIM alone is an incomplete perceptual measure, and wall-clock timings reflect this specific hardware configuration. A formal survey, additional image pairs, and a joint sweep of content and style weights would all strengthen what is reported here.

6 Conclusion (363 words)

This project implemented the optimisation-based neural style transfer method of Gatys, Ecker, and Bethge (2016) and investigated three design dimensions through a controlled experimental harness: backbone choice, loss weighting, and optimiser selection.

Across Research Question 1, layer set selection had a more pronounced effect on output character than the choice of backbone. The deep `_only` configurations produced outputs that retained photographic structure rather than transferring painterly texture, while the default and shallow `_only` sets produced the colour and brushstroke transfer the method is designed for. VGG16 and VGG19 produced comparable results at the same layer set, with VGG16 running faster at every configuration.

Research Question 2 showed that style weight is the more direct lever on artistic character. Increasing it produced a continuous shift from content-dominant to style-dominant outputs across the range tested. Total variation weight had a subtler effect on visual output, though at its highest tested value the TV loss dominated the total in a pattern consistent with the weight sensitivity described in Section 4.4.

Research Question 3 found that SGD and Adam reach qualitatively distinct outputs under identical settings on this image pair. Adam reached a substantially lower style loss but at a marked reduction in content structure; SGD distributed the loss more evenly and retained greater photographic detail. Wall-clock time was equivalent between the two optimisers.

The gallery confirmed that the pipeline generalises across content and style pairings beyond the baseline image pair used throughout the research question sweeps.

The findings carry two practical implications. Layer configuration deserves explicit attention when adapting the Gatys method to new images or frameworks, as

default settings from reference implementations may not transfer without recalibration. Optimiser choice affects the character of the output and can be treated as a creative parameter rather than a purely technical one.

Limitations include the use of a single image pair for the backbone and optimiser comparisons, the absence of a formal viewer survey, and the reliance on SSIM as the sole perceptual measure. Future work could address these through additional image pairs, a controlled survey, a joint sweep of content and style weights, and an investigation of feed-forward alternatives for applications where runtime is a constraint.

References

- [1] François Chollet. *Deep Learning with Python*. Shelter Island, NY: Manning Publications, 2018.
- [2] Yongcheng Jing et al. “Neural Style Transfer: A Review”. In: *IEEE Transactions on Visualization and Computer Graphics* 26.11 (2019), pp. 3365–3385.
- [3] Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge. “Image Style Transfer Using Convolutional Neural Networks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2414–2423.
- [4] Saleema Amershi et al. “Power to the People: The Role of Humans in Inter-active Machine Learning”. In: *AI Magazine* 35.4 (2014), pp. 105–120.
- [5] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2015.

-
- [6] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Net-works for Large-Scale Image Recognition”. In: *arXiv preprint arXiv:1409.1556* (2014).
 - [7] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. “Perceptual Losses for Real-Time Style Transfer and Super-Resolution”. In: *European Conference on Computer Vision (ECCV)*. Cham: Springer, 2016, pp. 694–711.
 - [8] Xun Huang and Serge Belongie. “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalisation”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 1501–1510.
 - [9] Yijun Li et al. “Universal Style Transfer via Feature Transforms”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017, pp. 386–396.
 - [10] Zhou Wang et al. “Image Quality Assessment: From Error Visibility to Structural Similarity”. In: *IEEE Transactions on Image Processing* 13.4 (2004), pp. 600–612.

Appendix A: Implementation Notebook

The full implementation notebook, including all code, cell outputs and logged results, is available at:

[View the notebook on GitHub](#)